

CSCD84: Artificial Intelligence

Assignment 3: Optimization and Deep Learning

Due Monday 11:59PM, Mar 17, 2025

1 Written Component

1.1 Implicit Regularization with SGD

Consider a regression problem where we are given N pairs of input-output pairs $D = \{\mathbf{x}_n, t_n\}_{n=1}^N$, where $\mathbf{x}_n \in \mathbb{R}^d$ is a d -dimensional feature and $t_n \in \mathbb{R}$ is the corresponding target. Suppose our function class is linear models $\mathcal{F} = \{f : \mathbb{R}^d \rightarrow \mathbb{R} \mid f(\mathbf{x}; \boldsymbol{\theta}) = \boldsymbol{\theta}^\top \mathbf{x}, \boldsymbol{\theta} \in \mathbb{R}^d\}$, and our loss function is the squared error $\ell(y, t) = \frac{1}{2}(y - t)^2$. Our goal is to find the empirical risk minimizer (ERM)

$$\hat{\boldsymbol{\theta}} = \arg \min_{\boldsymbol{\theta}} \mathcal{L}(D; \boldsymbol{\theta}) = \arg \min_{\boldsymbol{\theta}} \frac{1}{N} \sum_{n=1}^N \ell(\boldsymbol{\theta}^\top \mathbf{x}_n, t_n) = \arg \min_{\boldsymbol{\theta}} \frac{1}{2N} \|X\boldsymbol{\theta} - \mathbf{t}\|_2^2,$$

where $X = [\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N]^\top$ and $\mathbf{t} = [t_1, t_2, \dots, t_N]^\top$. Answer the following questions:

1. Show that $\hat{\boldsymbol{\theta}} = (X^\top X)^{-1} X^\top \mathbf{t}$. This is the closed-form solution of linear least squares! When the number of samples N and dimensionality d become large, it is infeasible to compute this—the inverse operation specifically is expensive! To this end, we should start thinking about using stochastic gradient descent (SGD) to optimize for our linear model.

Hint 1: Set the gradient of $\frac{1}{2N} \|X\boldsymbol{\theta} - \mathbf{t}\|_2^2$ to be zero and isolate for $\boldsymbol{\theta}$.

Hint 2: The gradient $\nabla_{\mathbf{x}} \mathbf{a}^\top \mathbf{x} = \mathbf{a}$, for $\mathbf{a}, \mathbf{x} \in \mathbb{R}^d$.

Hint 3: The gradient $\nabla_{\mathbf{x}} \mathbf{x}^\top A \mathbf{x} = (A + A^\top) \mathbf{x}$, for $\mathbf{x} \in \mathbb{R}^d, A \in \mathbb{R}^{d \times d}$.

2. Provide an example showing that when $d > N$, there can be infinite solutions—this is known as **overparameterization**.
3. Since there can be infinite solutions when learning an overparameterized (linear) model, it raises the question of what SGD converges to in this case. Perhaps quite interestingly, if we set the initial parameter $\boldsymbol{\theta}_0 = \mathbf{0}$ and assume that SGD does converge to a solution $\boldsymbol{\theta}_{SGD}$ such that $X\boldsymbol{\theta}_{SGD} = \mathbf{t}$, then the solution is actually the minimum-norm solution

$$\boldsymbol{\theta}_{SGD} = \arg \min_{\boldsymbol{\theta}} \|\boldsymbol{\theta}\|_2^2 \quad \text{satisfying} \quad \mathbf{t} = X\boldsymbol{\theta}.$$

Show that this is the case by answering the following:

- (a) Show that the SGD solution has the form $\boldsymbol{\theta}_{SGD} = \sum_{n=1}^N \alpha_n \mathbf{x}_n$ for some coefficients α_n . In other words, during SGD the parameter $\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \nabla_{\boldsymbol{\theta}} \mathcal{L}(D_t; \boldsymbol{\theta}_t)$, where η is the learning rate and D_t corresponds to the input-output pair sampled on the t 'th iteration, always lies in the span of the samples. We can also express the solution as $\boldsymbol{\theta}_{SGD} = X^\top \boldsymbol{\alpha}$.

Hint: Prove by induction on the iterates $\boldsymbol{\theta}_t$.

(b) Show that $\theta_{SGD} = X^\top (X X^\top)^{-1} \mathbf{t}$, which is the minimum-norm solution to the linear system $X\theta = \mathbf{t}$.

In general, we can expect other optimizers, loss functions, or models to have different implicit regularizers, but some non-linear models such as two-layered MLP with ReLU activations trained with SGD still converge to the minimum-norm solutions!

1.2 Networks with Skip Connections

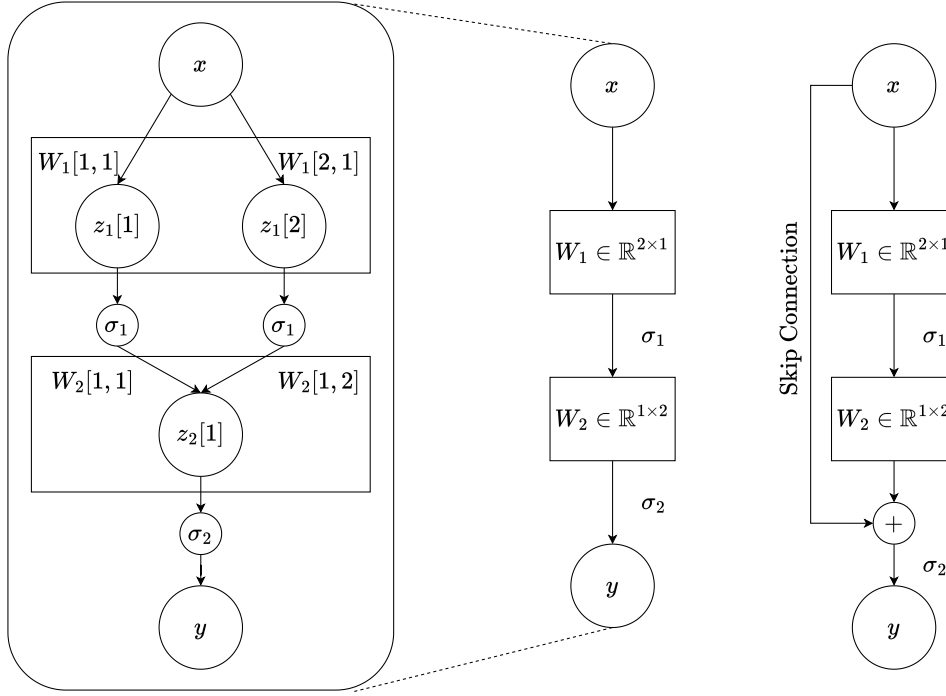


Figure 1: **Left:** A one-layered MLP f represented as the entire network. **Middle:** A one-layered MLP f with a higher-level representation. **Right:** An MLP with skip connection g with a higher-level representation.

Consider a binary-classification problem with one-dimensional feature $x \in \mathbb{R}$ and label space $\{0, 1\}$. You decided to apply a one-layered fully connected network (MLP) with two hidden units W_1 . The first hidden layer is followed by a non-linear activation function and the output layer is followed by a sigmoid function (see Figure 1, left). Mathematically we can write

$$y = f(x; W_1, W_2) = \sigma_2(W_2 \sigma_1(W_1 x)).$$

With deeper networks we can run into a problem known as **vanishing gradient** where the gradient in earlier layers becomes very small. This can happen when we have activation function with small gradient in general (e.g. sigmoid, tanh, etc.). To account for this, we can sum the output of second hidden layer (in this case, the output layer) with input x before applying the sigmoid. This addition is known as a **skip connection** that would preserve the gradient (see Figure 1, right). Networks with these skip connections are known as residual networks, which can be expressed as

$$y = g(x; W_1, W_2) = \sigma_2(W_2 \sigma_1(W_1 x) + x).$$

Answer the following questions:

1. Draw the computational graphs with each network f and g . For conciseness, write $W_k[i, j]$ to express the (i, j) 'th entry of matrix W_k and write $z_k[i]$ to express the i 'th hidden unit after applying W_k on its features.

2. Let t be the label, y be the prediction, and $\ell(y, t)$ be a loss function. Compute the derivative of ℓ with respect to x for each of f and g . For conciseness, write the gradient in terms of partial derivatives (e.g. $\frac{\partial \ell}{\partial x}$).
3. Using the derivatives above, provide an example where f experiences vanishing gradient (i.e. $\frac{\partial \ell}{\partial x} = 0$) whereas g does not suffer from this problem.

Hint: Define a poor activation in the first hidden layer.

Skip connections are widely used in deep learning nowadays (e.g. ResNet, GPT, and DDPM). Empirically, Li et al. (2018) have observed that using skip connections smooths out loss landscape (see Figure 2). Furusho and Ikeda (2020) corroborate this empirical finding through theoretical analysis (with certain assumptions, of course).

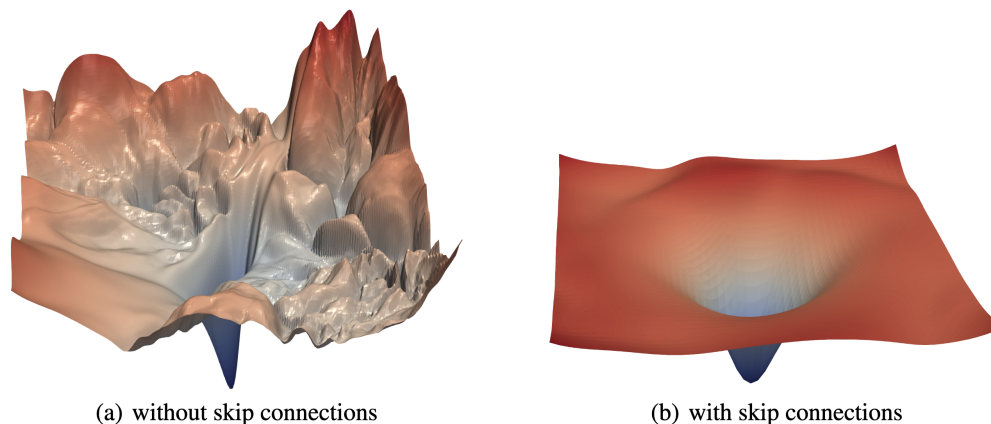


Figure 2: Loss landscape with **(b)** and without **(a)** skip connections (Li et al., 2018).

2 Programming Component

2.1 Python Packages and Imports

You will need to install:

```
1 $ pip install numpy
2 $ pip install torch
3 $ pip install torchvision
4 $ pip install matplotlib
```

We have already included all the necessary Python packages for this assignment—you are **not allowed to import any other packages**.

2.2 Tasks

In this assignment you have two tasks: implement MLPs from scratch and train multiple models on an image-classification task using PyTorch. The former will give you a better understanding on how neural networks can be trained in practice, whereas the latter will provide you hands-on experience on how to train and select models.

Files you will edit:

- `np_model.py`: A numpy-based MLP that implements prediction and backpropagation.
- `model.py`: Contains multiple neural network models.
- `compare_models.ipynb`: A notebook for comparing and visualizing different trained models.

Files you should read but can ignore:

- `main.py`: The entrypoint for training deep-learning models.
- `train.py`: The training pipeline—this includes performing model updates and checkpointing best models based on validation performance.

Implementing Backpropagation for MLPs with ReLU. In class, we discussed MLPs with L layers, each with $[N_1, N_2, \dots, N_L]$ hidden units respectively. The output $h_i \in \mathbb{R}^{N_i}$ of the i 'th layer is computed as

$$\begin{aligned} z_i &= W_i h_{i-1} + b_i \\ h_i &= \sigma_i(z_i) \end{aligned}$$

where $W_i \in \mathbb{R}^{N_i \times N_{i-1}}$ is the weight matrix, $b_i \in \mathbb{R}^{N_i}$ is the bias vector, and σ_i is the nonlinearity. We can show that

$$\frac{\partial \mathcal{L}}{\partial W_i} = \frac{\partial \mathcal{L}}{\partial z_i} h_{i-1}^\top, \quad \frac{\partial \mathcal{L}}{\partial b_i} = \frac{\partial \mathcal{L}}{\partial z_i}.$$

Your first task is to implement backpropagation for MLPs with ReLU activations—though, the backpropagation algorithm would work with other pointwise activations, e.g., Tanh, sigmoid, GeLU, anyway.

Implement the `compute_gradients` method in `np_model.py`, which computes the gradient w.r.t. all parameters. The loss that you will be using is the mean-squared error and is already implemented as the `mse` function. You may test the implementation by running `python np_model.py`. The code will compare the gradient computed by your code against the numerical gradient through finite difference. You can expect some numerical differences due to numerical stability—we will tolerate differences up to order of $1e-5$.

This backpropagation algorithm is essentially how we currently train most state-of-the-art models! In practice, autodiff is a little bit better designed, abstracting the implementation to have objects for each of variables, placeholders, operations, etc. Nevertheless, now that you understand how things work under the hood, let's train some deep-learning models for some tasks!

Image Classification on Fashion-MNIST. Your second task is to train a good deep-learning model that can classify images from the Fashion-MNIST dataset. The dataset corresponds to an image-classification task where the goal is to predict the type of apparel based on a 28×28 grayscale image. The dataset consists of 60,000 training examples and 10,000 test examples. Figure 3 shows ten examples of input-output pairs from the training set.

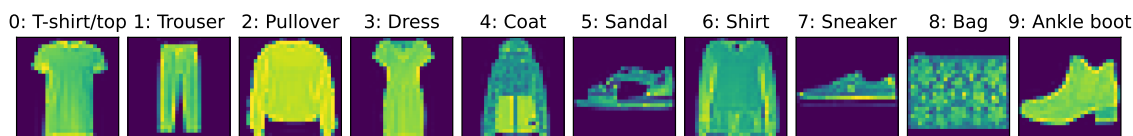


Figure 3: Examples from Fashion-MNIST.

The training code is already implemented for you, but what's missing is choosing the best hyperparameters. To this end, you have to train multiple models, select the best model, and evaluate it on the test examples. As mentioned in class, selecting the best model on test examples is cheating! Run `main.py` to train models:

```
1 # Default setting, use --help to see available arguments.
2 $ python main.py
3
4 # Use MLP or MLP with skip connection
5 $ python main.py --model=[MLP, SkipConnectionMLP]
6
7 # Use SGD or Adam
8 $ python main.py --optimizer=[SGD, Adam]
```

You can also change batch size, learning rate, number of training epochs.

Complete the following tasks:

1. Train various deep-learning models using `main.py`, include at most five different hyperparameter settings.
2. Choose the best hyperparameter setting and describe the rationale in `compare_models.ipynb`.
3. Evaluate the best model by turning on the `--eval` flag when running `main.py`, and report the evaluation performance.

It's no fun if you are simply running code without trying things yourself—implement your own custom models in the `CustomModel` class under `models.py`. So long as you can train the model with reasonable time using a CPU, you can use any architecture you want! You may use `--model=CustomModel` to run your model once you are satisfied with the implementation.

Grading. We will put 2 points based on the correctness of `np_model.py`. We will put 2 points based on the hyperparameter selection question and the evaluation results:

- 1/2 if your hyperparameter choice achieves between $[70, 80]\%$ evaluation accuracy
- 2/2 if your hyperparameter choice achieves above $(80, 100]\%$ evaluation accuracy

We will also give two bonus marks based on the `CustomModel`:

- 1/2 if your evaluation accuracy is top 10% of the class performance
- 2/2 if your evaluation accuracy is the best of the class performance

NOTE: We will be running your model to confirm the results, so don't try to make up numbers!

3 Submission Instructions

For the written component, please submit `A3.pdf`. For the programming component, please submit only `np_model.py`, `model.py`, `compare_models.ipynb`, and optionally your custom model's final parameters, named as `custom_model.pt`. We will evaluate your model by executing `python main.py --eval --load_path=<PATH>/custom_model.pt`.

NOTE 1: Please make sure your notebook has all the outputs presented.

NOTE 2: Please make sure your `custom_model.pt` is no larger than 50MB. Failing to satisfy this will exclude you from the competition!

References

- Li, H., Xu, Z., Taylor, G., Studer, C., & Goldstein, T. (2018). Visualizing the loss landscape of neural nets. *Advances in Neural Information Processing Systems*.
- Furusho, Y., & Ikeda, K. (2020). Theoretical analysis of skip connections and batch normalization from generalization and optimization perspectives. *APSIPA Transactions on Signal and Information Processing*.